

Djed: A Formally Verified Crypto-Backed Autonomous Stablecoin Protocol

Joachim Zahnentferner
Djed Alliance
Brazil
zahnentferner@gmail.com

Dmytro Kaidalov
Input Output
Ukraine
dmytro.kaidalov@iohk.io

Jean-Frédéric Etienne
Input Output
France
jean-frederic.etienne@iohk.io

Javier Díaz
Input Output and Atix Labs
Argentina
jdiaz@atixlabs.com

Abstract—This paper describes Djed, a stablecoin protocol that behaves like an autonomous central bank that buys and sells its currency (the stablecoin) for a price in a range that is pegged to a target price. It is crypto-backed in the sense that the bank keeps a volatile cryptocurrency in its reserve. The reserve is used to buy stablecoins from users that want to sell them. And revenue from sales of stablecoins to users is stored in the reserve. Besides stablecoins, the protocol also trades reservecoins in order to capitalize itself and maintain a reserve ratio significantly greater than one. To the best of our knowledge, this is the first implemented stablecoin protocol where stability claims are precisely and mathematically stated and proven. Furthermore, the claims and their proofs are formally verified using two different techniques: bounded model checking, to exhaustively search for counterexamples to the claims or to establish invariant satisfaction; and interactive theorem proving, to build rigorous formal proofs using a proof assistant with automated theorem proving features.

Index Terms—blockchain, cryptocurrency, stablecoin

I. INTRODUCTION

In the narrowest sense, a *stablecoin* is a cryptocurrency that has its price pegged to a fiat currency (e.g. USD) and is fully backed by reserves denominated in the same fiat currency. More broadly, a *stablecoin* can be defined as a digital asset that has mechanisms to maintain a low deviation of its price from a target price. Since the invention of Bitcoin, a major obstacle for the wider adoption and recognition of cryptocurrencies as proper currencies, particularly as means of exchange and units of account, has been the instability of their price in relation to fiat currencies. Stablecoins aim to overcome this obstacle.

Various mechanisms can contribute to a stablecoin's stability. Ultimately, they are all grounded on the basic economics of supply and demand. If demand for buying/selling stablecoins is higher than the current supply of sale/purchase orders, this supply must be increased to avoid a price increase/decrease.

Backing the stablecoins by reserves and using these reserves to actively buy and sell stablecoins for prices close to the target price is currently the most common mechanism. This mechanism is not exclusive to stablecoins; it can be seen in pegged national currencies such as the HKD (Hong Kong Dollar). Centrally operated fiat-pegged fiat-backed stablecoins such as USDT and USDC use variations of this mechanism

that may differ, for instance, on the actual composition of the reserves and with whom the operator interacts to buy and sell stablecoins. As long as the stablecoin is fully backed by reserves in the currency to which it is pegged and the operator can react quickly enough to variations in the demand, it is easy to see that stability will be guaranteed. Typically, the reserves will not be kept all in cash in a vault, safe or bank account, but rather at least partly in interest-bearing financial instruments such as bonds. The returns from such investments provide revenue for the operator. The risks associated with these investments may cause the stablecoin to eventually lose its full-backing, compromising the stability in the long-term. Lack of liquidity of these investments may cause the operator to be unable to react quickly enough to changes in demand, compromising the stability in the short-term. Another source of revenue are the fees or spread practiced when trading the stablecoin. For example, if the operator sells a USD-pegged stablecoin for 1.005 USD and buys it for 0.995 USD, it has a revenue of 1 cent for every stablecoin that it buys and then sells, while keeping the price stable within the range from 0.995 and 1.005.

The main drawback of fiat-backed stablecoins is the required trust on the entities keeping the reserves. This is not only a theoretical concern. Skepticism about the full-backing combined with insufficient stabilization measures have actually already caused USDT and USDC to trade at discounts of around 10%.

Interestingly, such reserve transparency issues do not arise when the backing asset is a cryptocurrency on a public blockchain. Furthermore, the inefficient and unreliable execution of stabilization measures can be eliminated by implementing the stabilization mechanisms as smart contracts that are automatically, reliably and transparently executed.

This paper describes Djed: a crypto-backed autonomous stablecoin protocol. Djed acts as an autonomous bank, keeping a *reserve* R of *BaseCoins* (BCs), and minting and burning *StableCoins* (SCs) and *ReserveCoins* (RCs). It maintains the peg of the SCs to a target price by buying and selling SCs, using its reserve. While doing so, it charges fees and accumulates them in its reserve. The beneficiaries of this revenue stream are the RC holders, who contribute with additional funds to the reserve and take the risk of price fluctuation.

Djed is defined in Section II. The protocol is designed to be as intuitive and minimalistic as possible, while still being

stable, safe and secure. Several stability properties are stated and proven as theorems in Section III. Sections IV and V describe and discuss the formal verification of the theorems and proofs using, respectively, model checking and interactive theorem proving techniques. Current implementations of Djed are briefly discussed in Section VI. Section VII discusses related work, with a focus on three algorithmic stablecoins that were influential in the design of Djed.

II. THE DJED PROTOCOL

The Djed protocol's main purpose is to maintain the price of internally issued stablecoins (SCs) stable relative to a target price. The target price can be, for instance, the price of USD or EUR, or any other currency, asset or index. To achieve this, Djed maintains a reserve of basecoins (BCs), which is another cryptocurrency, such as BTC, ETH or ADA. The protocol issues SCs to users in exchange for BCs, which are then stored in the reserve. The amount of BCs a user has to pay for one stablecoin is defined by the exchange rate between the BC and the target asset, which is provided by an oracle (e.g., if a user buys 1 SC pegged to USD, the user pays an amount of BCs worth 1 USD). Similarly, the SCs can be sold back to Djed in exchange for a corresponding amount of BCs according to the current exchange rate.

As the price of BCs can be volatile, the protocol aims to have abundant reserves to preserve the peg to the target price even if the price of BCs drops significantly relative to the target asset. Additional BCs are acquired to the reserve through issuance of reservecoins (RCs). Purchase of RCs is attractive for users looking for greater returns. Not only RC holders indirectly benefit from the trading fees that accumulate in the reserve, but they are also exposed to a leverage effect: *ceteris paribus*, when the price of BCs grows, the price of RCs grows even more. Furthermore, the lower the reserve ratio, the stronger this leverage effect. Thus, buying reservecoins is more attractive precisely when the reserve ratio is lower. As buying reservecoins increases the reserve ratio, this varying leverage effect serves as an important mechanism for the sustainability of the protocol. On the other hand, the leverage effect also brings greater risks: *ceteris paribus*, when the price of BCs fall, the price of RCs fall even more. Thus, RC holders take the risk of BC price depreciation.

Djed's state is a triple (R, N_{SC}, N_{RC}) where:

- R is the number of basecoins (BCs) in Djed's reserve,
- N_{SC} is the number of stablecoins (SCs) in circulation,
- N_{RC} is the number of reservecoins (RCs) in circulation.

The protocol defines 4 operations to interact with the users:

- 1) The user **Buys SCs** from Djed by sending an amount of BCs to Djed. Djed then stores the provided BCs in the reserve and mints new SCs to the user.
- 2) The user **Sells SCs** to Djed. Djed then burns the provided SCs and returns to the user an amount of BCs from the reserve.

- 3) The user **Buys RCs** from Djed by sending an amount of BCs to Djed. Djed then stores the provided BCs in the reserve and mints new RCs to the user.
- 4) The user **Sells RCs** to Djed. Djed then burns the provided RCs and returns to the user an amount of BCs from the reserve.

The *target price* of an SC in BCs is denoted P_{SC}^t . For example¹, in the case of a stablecoin pegged to some *peg currency* PC (e.g. EUR, USD, ...), $P_{SC}^t = X_{BC}^{PC}$ BC where X_{BC}^{PC} is the price of 1 unit of the peg currency in BCs.

Because Djed's reserve may be insufficient to buy back all stablecoins for the target price, Djed sets the actual *price* P_{SC} of SCs according to the following equation:

$$P_{SC} = \begin{cases} P_{SC}^t & \text{if } N_{SC} = 0 \\ \min(P_{SC}^t, \frac{R}{N_{SC}}) & \text{otherwise.} \end{cases} \quad (1)$$

The portion of Djed's reserve that would need to be used to buy back all stablecoins is known as its *liabilities*:

$$L(N_{SC}) = N_{SC} P_{SC} \quad (2)$$

Because P_{SC} is volatile, Djed strives to keep a high *reserve ratio*:

$$r(R, N_{SC}) = \frac{R}{L(N_{SC})} \quad (3)$$

Djed does so by having a *minimum reserve ratio* r_{min} and disallowing users from buying SCs or selling back RCs if, after these actions, $r(R, N_{SC}) < r_{min}$. To prevent dilution for the RC holders, Djed also has a *maximum reserve ratio* r_{max} and disallows users from buying more RCs if, before or after the purchase, $r(R, N_{SC}) > r_{max}$; unless $N_{SC} < N_{SC}^*$, where N_{SC}^* is the *threshold number of stablecoins* parameter². Note that, even though Djed disallows some types of purchases and sales when the reserve ratio is above the maximum or below the minimum, the reserve ratio may still go above the maximum or below the minimum due to price fluctuations.

The reserve surplus is Djed's *equity*:

$$E(R, N_{SC}) = R - L(N_{SC}) \quad (4)$$

Djed's equity is shared equally among RC holders, and thus the *target price* of RCs is:

$$P_{RC}^t(R, N_{SC}, N_{RC}) = \frac{E(R, N_{SC})}{N_{RC}} \quad (5)$$

However, the target price is undefined when $N_{RC} = 0$ and a price equal to 0 when $E(R, N_{SC}) = 0$ would be problematic, because users would be able to buy an arbitrary number of

¹Djed does not need to be pegged to a fiat currency. It just needs a target price. The target price could be a weighted average of the price of a volatile asset, a stock index, an inflation index, ...

²The main purpose of this parameter is to allow users to buy reservecoins, and hence contribute to the quick growth of the reserve, soon after initialization. If it were not for this, it is easy to see that users would be able to buy neither stablecoins nor reservecoins right after initialization when $N_{SC} = 0$ and $N_{RC} = 0$. Moreover, this parameter also allows a quick re-initialization in the unlikely event that all stablecoins and reservecoins are sold back and a state where $N_{SC} = 0$ and $N_{RC} = 0$ is reached again.

RCs without any cost. Therefore, Djed sets the actual *buying price* P_{RC}^b according to the following equation:

$$P_{RC}^b(R, N_{SC}, N_{RC}) = \begin{cases} \max(P_{RC}^t, P_{RC}^{min}) & \text{if } P_{RC}^t \text{ is defined} \\ P_{RC}^{min} & \text{otherwise} \end{cases} \quad (6)$$

where P_{RC}^{min} is a parameter of Djed.

The pricing of Djed's operations is summarized in Table I.

TABLE I
DJED'S OPERATIONS

Operation	User Sends	User Receives	Invariant ^{a,b}
Buy SCs	$n(1+fee)P_{SC}$	n SCs	$r(R, N_{SC}) \geq r_{min}$
Sell SCs	n SCs	$n(1-fee)P_{SC}$	
Buy RCs	$n(1+fee)P_{RC}^b$	n RCs	$r(R, N_{SC}) \leq r_{max}$ or $N_{SC} < N_{SC}^*$
Sell RCs	n RCs	$n(1-fee)P_{RC}^t$	$r(R, N_{SC}) \geq r_{min}$

^aThe invariants must hold *before* and *after* the actions.

^bThe invariants in the table are stated in a way to ease readability and intuitive understanding. However, note that the calculation of the reserve ratio potentially involves a division by zero. Implementations should circumvent this issue by using the standard technique of multiplying both sides of the inequation by the denominator of the reserve ratio. Thus, for instance, the invariant $r(R, N_{SC}) \geq r_{min}$ should be understood and implemented as $R \geq L(N_{SC})r_{min}$.

Djed's 5 parameters (r_{min} , r_{max} , fee , N_{SC}^* , P_{RC}^{min}) are assumed to be set to values that satisfy the following constraints: $r_{min} > 1 + fee$; $r_{max} \geq r_{min}$; $0 < fee \leq 1$; $N_{SC}^* > 0$; $P_{RC}^{min} > 0$.

Djed also depends on one (and only one) external variable: the oracle exchange rate X_{BC}^{PC} . The models and formalizations assume that $X_{BC}^{PC} > 0$. This is a reasonable assumption, because $X_{BC}^{PC} = 0$ (or, worse, $X_{BC}^{PC} < 0$) would imply that the value of the peg currency has collapsed. In such cases, a stablecoin pegged to it would be worthless anyway.

III. STABILITY PROPERTIES

Djed enjoys several stability properties. The first one is that, in the normal reserve ratio range where purchases and sales are not restricted³, users have no incentive to trade stablecoins outside the peg range in a secondary market.

Theorem 1 (Peg Maintenance - Upper Bound). *If $r(R, N_{SC}) > r_{min} + \epsilon$ (for a sufficiently large ϵ) and a user u wants to sell a stablecoin in the secondary market for a price P such that $P > (1 + fee)P_{SC}^t$, then there is no rational user u^* who would buy from u .*

Proof: Assume, for the sake of contradiction, that such a rational u^* exists. Then u^* would have had two options: to buy a stablecoin from u for P ; or to buy a stablecoin directly from the bank for P' where $P' = (1 + fee)P_{SC}$. Note that the second option is available because, for a sufficiently large ϵ , the invariant for the action of buying stablecoins from the bank holds. By definition of P_{SC} , $P' < (1 + fee)P_{SC}^t$. Therefore,

³In practice, there are other factors beyond the reserve ratio that may restrict interaction of the user with the bank, such as blockchain congestion and high blockchain transaction fees. Such factors would have an effect on the ability of the bank to maintain the peg range in secondary markets.

buying directly from the bank for P' would have been less costly than buying from u for P , and would have been the preferred option for a rational user. Hence u^* is irrational. ■

Theorem 2 (Peg Maintenance - Lower Bound). *If $r(R, N_{SC}) > 1$ and a user u wants to buy a stablecoin in the secondary market for a price P such that $P < (1 - fee)P_{SC}^t$, then there is no rational user u^* who would sell to u .*

Proof: Assume, for the sake of contradiction, that such a rational u^* exists. Then u^* would have had two options: to sell a stablecoin to u for P ; or to sell a stablecoin directly to the bank for P' where $P' = (1 - fee)P_{SC}$. By definition of P_{SC} and the fact that $r(R, N_{SC}) > 1$, $P' = (1 - fee)P_{SC}^t$. Therefore, selling directly to the bank for P' would have been more profitable than selling to u for P , and would have been the preferred option for a rational user. Hence u^* is irrational. ■

The second stability property is that stablecoins remain pegged despite market crashes up to a magnitude that depends on the reserve ratio.

Theorem 3 (Peg Robustness during Market Crashes). *If $N_{SC} > 0$, r is the current reserve ratio, x and x' are the exchange rates before and after the crash, $r > 1$, then the bank can tolerate a basecoin price crash of $\frac{r-1}{r}$ and the new stablecoin price would still be P_{SC}^t .*

Proof: By the definition of P_{SC} , the peg is maintained as long as $P_{SC}^t \leq \frac{R}{N_{SC}}$. By the definition of P_{SC}^t , reserve ratio and liabilities, this inequation can be simplified to $x' \leq rx$. Since x and x' are the exchange rates of 1 unit of pegged currency in BCs, the exchange rates of 1 BC in pegged currency are $y = \frac{1}{x}$ and $y' = \frac{1}{x'}$. The inequation can then be rewritten as $y' \geq \frac{y}{r}$. Therefore, $\frac{y-y'}{y} \leq \frac{r-1}{r}$. ■

Another crucial property is that the bank never becomes insolvent, which is a condition defined by having negative equity⁴.

Theorem 4 (No Insolvency). *In all bank states and for any exchange rate, $E(R, N_{SC}) \geq 0$.*

Proof: By definition, $E(R, N_{SC}) = R - L(N_{SC}) = R - N_{SC}P_{SC}$. By the definition of P_{SC} , there are 3 cases to consider:

- $N_{SC} = 0$: in this case, $E(R, N_{SC}) = R - 0P_{SC}^t = R$ and $R \geq 0$.
- $N_{SC} \neq 0$ and $\frac{R}{N_{SC}} \leq P_{SC}^t$: in this case, $E(R, N_{SC}) = R - N_{SC}\frac{R}{N_{SC}} = 0$.
- $N_{SC} \neq 0$ and $\frac{R}{N_{SC}} > P_{SC}^t$: in this case, $E(R, N_{SC}) = R - N_{SC}P_{SC}^t$. Since $\frac{R}{N_{SC}} > P_{SC}^t$, it must be the case that $E(R, N_{SC}) > R - N_{SC}\frac{R}{N_{SC}}$. Therefore, $E(R, N_{SC}) > 0$. ■

Another important property is that, provided that the exchange remains constant, the bank is never in a state susceptible to

⁴Rather than promise the impossible and fail, Djed adjusts the stablecoin price function fairly, promising to pay back only what it can, being liable only for what it can be liable for. The significance of the proof of solvency is that it shows that Djed is carefully designed not to promise more than what can be fulfilled.

bank runs where stablecoin holders would feel incentivized to race against each other to sell their stablecoins.

Theorem 5 (No Bank Runs for Stablecoins). *Let u_1 and u_2 be two stablecoin holders. Let P_{SC}^1 be the price of stablecoins obtained by u_1 by selling its stablecoins back to the bank. Let P_{SC}^2 be the price of stablecoins obtained by u_2 by selling its stablecoins back to the bank. Assume, without loss of generality, that u_2 's sale occurs after u_1 's sale. Then, in all bank states and for any constant exchange rate, $P_{SC}^2 \geq P_{SC}^1$.*

Proof: Firstly, note that the sale of stablecoins back to the bank is never restricted by the invariant. Therefore, it cannot be the case that the sale by u_1 would bring the bank to a state where u_2 's sale would be blocked. Let q be the quantity of stablecoins sold by u_1 and R_1 be the reserve after the sale by u_1 . Then, following the definition of P_{SC} , there are two cases to consider:

- $P_{SC}^1 = P_{SC}^t$: in this case, $P_{SC}^t \leq \frac{R}{N_{SC}}$ and $R_1 = R - q(1 - fee)P_{SC}^t$. Then:

$$P_{SC}^2 = \min \left(P_{SC}^t, \frac{R - q(1 - fee)P_{SC}^t}{N_{SC} - q} \right).$$

Using the fact that $P_{SC}^t \leq \frac{R}{N_{SC}}$, we have that:

$$\frac{R - q(1 - fee)P_{SC}^t}{N_{SC} - q} \geq \frac{R - q(1 - fee)\frac{R}{N_{SC}}}{N_{SC} - q}.$$

Simplifying, we obtain:

$$\frac{R - q(1 - fee)P_{SC}^t}{N_{SC} - q} \geq \frac{R}{N_{SC}} \frac{N_{SC} - q + qfee}{N_{SC} - q}.$$

And hence:

$$\frac{R - q(1 - fee)P_{SC}^t}{N_{SC} - q} \geq \frac{R}{N_{SC}}.$$

Therefore, $P_{SC}^2 = P_{SC}^t = P_{SC}^1$.

- $P_{SC}^1 = \frac{R}{N_{SC}}$: in this case, $P_{SC}^t \geq \frac{R}{N_{SC}}$ and $R_1 = R - q(1 - fee)\frac{R}{N_{SC}}$. Then:

$$P_{SC}^2 = \min \left(P_{SC}^t, \frac{R - q(1 - fee)\frac{R}{N_{SC}}}{N_{SC} - q} \right).$$

We now have two cases:

- $P_{SC}^2 = P_{SC}^t$: in this case, $P_{SC}^2 > P_{SC}^1$.
- $P_{SC}^2 = \frac{R - q(1 - fee)\frac{R}{N_{SC}}}{N_{SC} - q}$: in this case, simplifying, we have $P_{SC}^2 = \frac{R}{N_{SC}} \frac{N_{SC} - q + qfee}{N_{SC} - q}$ and therefore $P_{SC}^2 \geq P_{SC}^1$.

The following theorem shows that, provided that the exchange rate remains constant, the equity per reservecoin increases after every operation due to collected fees. Note that this is one of the main incentives to buy and hold RCs.

Theorem 6 (Monotonically Increasing Equity per Reservecoin). *Assuming that the exchange rate remains constant and $N_{RC} > 0$, for every action a , $\frac{E(R^a, N_{SC}^a)}{N_{RC}^a} \geq \frac{E(R, N_{SC})}{N_{RC}}$, where R^a , N_{SC}^a , $N_{RC}^a > 0$ are respectively, the reserve, number of stablecoins and number of reservecoins after action a .*

Proof: See Appendix A for the proof. ■

As a corollary, the bank is therefore never subject to *draining* of its reserves by malicious users, provided that the exchange rate remains constant.

Theorem 7 (No Reserve Draining). *Assuming that the exchange rate remains constant, for any initial bank state $(R^0, N_{SC}^0, N_{RC}^0)$, there is no sequence of actions $a_1, a_2, \dots, a_n$ that would bring the bank to a state $(R^n, N_{SC}^n, N_{RC}^n)$ such that $R^n < R^0$, $N_{SC}^n = N_{SC}^0$ and $N_{RC}^n = N_{RC}^0$, assuming $N_{SC}^i > 0, N_{RC}^i > 0$ for all $i \in \{0..n-1\}$.*

Proof: We will consider two separate cases:

- If $E(R^0, N_{SC}^0) > 0$: assume, for the sake of contradiction, that such a sequence exists. Then note that the equity per reservecoin after the sequence of actions would be smaller than the equity per reservecoin before the sequence of actions. Therefore, the assumption leads to a contradiction with Theorem 6.
- If $E(R^0, N_{SC}^0) = 0$: note that in this case $P_{SC}^0 = \frac{R^0}{N_{SC}^0}$. Let's first prove that for any action a it holds that $\frac{R^a}{N_{SC}^a} \geq \frac{R^0}{N_{SC}^0}$. There are 4 different actions to consider:

- *Sell RCs* and *Buy SCs* are disallowed if $E(R^0, N_{SC}^0) = 0$ (due to $r < r_{min}$).
- *Buy RCs*: in this case $R^a > R^0$ and $N_{SC}^a = N_{SC}^0$, hence $\frac{R^a}{N_{SC}^a} > \frac{R^0}{N_{SC}^0}$ holds.
- *Sell SCs*: in this case $R^a = R^0 - n(1 - fee)\frac{R^0}{N_{SC}^0}$, where n is the number of sold SCs, hence: $\frac{R^0 - n(1 - fee)\frac{R^0}{N_{SC}^0}}{N_{SC}^0 - n} \geq \frac{R^0}{N_{SC}^0}$, $\frac{N_{SC}^0 - n(1 - fee)}{N_{SC}^0 - n} \geq 1$, which holds due to $n(1 - fee) \leq n$.

Now, let assume, for the sake of contradiction, that there is a sequence $a_1, a_2, \dots, a_n$ such that $R^n < R^0$, $N_{SC}^n = N_{SC}^0$ and $N_{RC}^n = N_{RC}^0$. In this case $\frac{R^n}{N_{SC}^n} < \frac{R^0}{N_{SC}^0}$, which contradicts the just proved statement.

Note that if the sequence contains $a_i, i \in [1, n]$ such that equity becomes $E(R^i, N_{SC}^i) > 0$, any further $a^j, j > i$ would make it zero again as it contradicts Theorem 6. Hence, $R^n < R^0$ and $N_{SC}^n = N_{SC}^0$ is impossible in this case due to $E(R^n, N_{SC}^n) > E(R^0, N_{SC}^0)$. ■

And, finally, the dilution to which reservecoin holders may be subject is bounded.

Theorem 8 (Bounded Dilution). *Assuming that $N_{SC} \geq N_{SC}^*$ and provided that the exchange rate and the number of stablecoins $N_{SC} > 0$ remains constant, if P_{RC}^b is the current price of reservecoins, $N_{RC} > 0$ is the current number reservecoins and R is the current reserve, then the maximum number of RCs that can be bought additionally is $\frac{r_{max} N_{SC} P_{SC}^t - R}{(1 + fee) P_{RC}^b}$ (or 0 if this number is negative).*

Proof: Let's consider an operation a to buy n reservecoins such that $r^a = r_{max}$ is the reserve ratio after the operation. Given that $r^a = \frac{R^a}{N_{SC} P_{SC}^a}$, where $R^a = R + n(1 + fee)P_{RC}^b$, we can find the maximum number n of RCs to be bought from the equation $\frac{R + n(1 + fee)P_{RC}^b}{N_{SC} P_{SC}^a} = r_{max}$. Provided that

$r^a = r_{max} > 1$ it follows that $P_{SC}^a = P_{SC}^t$ and, thus, $n = \frac{r_{max} N_{SC} P_{SC}^t - R}{(1+fee)P_{RC}^b}$. Note that splitting operation a into two (or more) operations will not yield more reservecoins as the price P_{RC}^b monotonically grows for every operation (see Theorem 6). ■

IV. MODEL-CHECKING DJED'S STABILITY PROPERTIES

The KIND2 SMT-based model checker [1] was used to validate the stability properties of the stablecoin contract. It takes as input one or more LUSTRE files [2] annotated with the properties to be verified and outputs whether each property is satisfied or not. In particular, Bounded Model Checking (BMC) [3] is used for detecting any property violation. In particular, BMC determines whether a counterexample at a given discrete time step k exists for a property P by verifying the following propositional formula:

$$bmc(k) \equiv I(\vec{x}_0) \wedge \bigwedge_{i=0}^{k-1} (C(\vec{x}_i) \wedge T(\vec{x}_i, \vec{x}_{i+1})) \wedge C(\vec{x}_k) \wedge \neg P(\vec{x}_k)$$

Here, $\vec{x}$ represents the set of state variables. Notation $\vec{x}_0, \dots, \vec{x}_k$ is used to represent vector copies of $\vec{x}$ at each time step $i \in [0..k]$. Quantifier-free formulae $I(\vec{x}_0)$ and $C(\vec{x}_i)$ respectively denote the set of initial states and any assumption imposed on $\vec{x}$. Formula $T(\vec{x}_i, \vec{x}_{i+1})$ represents the transition relation between $\vec{x}_i$ and $\vec{x}_{i+1}$ over one time step. Finally, formula $P(\vec{x}_k)$ denotes the set of properties to be satisfied.

For invariant satisfaction, a proof by induction strategy called K-Induction is used [4], whereby a base case and an induction step must be proved for a property P to be considered as an invariant of the system. If the base case fails then a concrete counterexample exists for P . Induction at a given discrete time step k is characterized by the following formulae:

$$\begin{aligned} base(k) &\equiv \text{for } 0 \leq d \leq k \\ &I(\vec{x}_0) \wedge \bigwedge_{i=0}^{d-1} (C(\vec{x}_i) \wedge T(\vec{x}_i, \vec{x}_{i+1})) \wedge C(\vec{x}_d) \rightarrow P(\vec{x}_d), \\ step(k) &\equiv \bigwedge_{i=0}^{k-1} (P(\vec{x}_i) \wedge C(\vec{x}_i) \wedge T(\vec{x}_i, \vec{x}_{i+1})) \wedge C(\vec{x}_k) \rightarrow P(\vec{x}_k). \end{aligned}$$

It can be observed that the base case is verified from time step 0. The step case is quite similar to $base(k)$, except that the initial constraint (i.e., $I(\vec{x}_0)$) is removed and property P is assumed to hold on all states up to time step $k-1$ ⁵.

A. Data Types

The following data types are used to specify the input, output and state variables maintained by the stablecoin contract in LUSTRE.

⁵Note that formula $step(k)$ is reduced in proving whether $P(\vec{x}_0)$ can solely be inferred from $C(\vec{x}_0)$ when $k = 0$, i.e., the proof by induction really starts when $k = 1$.

Listing 1: StableCoin Data Types

```
type Order = enum { MintSC, MintRC, NoOrder };
type Proceed = enum { MintedSC, MintedRC, RedeemedSC,
  RedeemedRC, Error, NoReply };
type ErrorInfo = enum { Min_Ratio_Violated, Max_Ratio_Violated,
  Invalid_Mint_Value, None };
type InputMsg = struct { order: Order; qnt: AmountType };
type OutputMsg = struct { ack: Proceed; err: ErrorInfo;
  price: ReserveType };
type Parameters = struct { r_min: RatioType; r_max: RatioType;
  fee: ReserveType; n_sc_s: N_Type;
  p_min: ReserveType };
function imported params () returns (out: Parameters);
```

The input to the contract is defined as a structure specifying the type of order submitted to the contract and the number of coins to be bought (i.e., positive value) or sold (i.e., negative value). The contract also produces a structure as output to indicate whether or not an order was processed successfully. On a successful order, the `price` field indicates the actual buying price (i.e., value increasing reserve) or selling price (i.e., value decreasing reserve). Type `Parameters` specifies the parameters characterizing the Djed specification and abstract function `params` (i.e., imported directive) is introduced to provide a global access to parameters.

B. Infinite-State System Specification

Listing 2 illustrates how the stablecoin state-transition system is specified in LUSTRE. In LUSTRE, the notion of *node* is used to model transition systems, using specific temporal combinators to characterize stateful behaviours. Function definitions are also supported in LUSTRE. However, they should be stateless.

Listing 2: StableCoin State-Transition System

```
node StableCoin(i_msg: InputMsg; rate: RateType)
  returns (p_reserve: ReserveType; p_sc: N_Type; p_rc: N_Type;
    reserve: ReserveType; n_sc: N_Type; n_rc: N_Type;
    o_msg: OutputMsg)
let
  p_reserve = 0 -> pre reserve;
  p_sc = 0 -> pre n_sc;
  p_rc = 0 -> pre n_rc;
  o_msg = if i_msg.order = NoOrder then
    NullReply
  else if i_msg.order = MintSC then
    mintSC(i_msg.qnt, rate, p_reserve, p_sc)
  else
    mintRC(i_msg.qnt, rate, p_reserve, p_sc, p_rc,
      t_bootstrap);
  reserve = if o_msg.ack = Error
    then p_reserve else p_reserve + o_msg.price;
  n_sc = if o_msg.ack = MintedSC or o_msg.ack = RedeemedSC
    then p_sc + i_msg.qnt else p_sc;
  n_rc = if o_msg.ack = MintedRC or o_msg.ack = RedeemedRC
    then p_rc + i_msg.qnt else p_rc;
tel
```

The stablecoin node takes two arguments as input, namely: the user's order and the exchange rate. Operator `->` ("followed by") is used to initialize the state variables, while operator `pre` makes reference to the previous value of an expression. Variables `reserve`, `n_sc` and `n_rc` respectively correspond to the bank reserve, number of stablecoins in circulation and number of reservecoins in circulation. They are updated only when an order is successfully processed. Otherwise, they retain their previous value.

The minting and burning logics for both stablecoins and reservecoins are respectively specified in functions `mintSC` and `mintRC`. Unlimited buying and selling of reservecoins are authorized when there are no stablecoins in circulation. Moreover, minting and burning actions are not authorized

when the exchange rate is not greater than zero. Selling of actions are also forbidden when the number of coins being sold is greater than the number in circulation.

C. Property Specification

Each stability property is formalized in a specific LUSTRE node. Properties about the stablecoin contract are expressed in a black-box fashion (i.e., only formulated in terms of inputs and outputs of the contract). Directive `check` is used to specify properties to be verified. The `assert` directive is used to specify assumptions under which properties hold. The two main assumptions concern the contract's parameters, namely: parameters remain constant from one time step to the next; and parameters should satisfy the min and max values necessary to guarantee stability.

Listing 3 shows how Theorem 5 is formally specified. Note that the LUSTRE formalization is even stronger than the formulation in Section III, as it also covers situations where any arbitrary number of actions between two successful selling SC events can occur. In the formalization, property "THEOREM_5" states the following:

If the exchange rate remains constant and at least one SC was already sold then, on a successful selling order, the current selling price per SC is greater than or equal to the previously applied price.

Listing 3: Theorem 5 Formalization

```
node Theorem5 (i_msg: InputMsg; rate: RateType)
  returns (o_msg: OutputMsg)
var reserve: ReserveType; n_sc: N_Type; n_rc: N_Type;
p_reserve: ReserveType; p_sc: N_Type; p_rc: N_Type;
constant_rate: bool; p_seller: SCSellerType;
c_seller: SCSellerType;
let
  p_reserve, p_sc, p_rc, reserve, n_sc, n_rc, o_msg =
    StableCoin(i_msg, rate);
  -- Assuming that parameters remain constant
  assert true ->
    params().r_min = pre params().r_min and
    params().r_max = pre params().r_max and
    params().fee = pre params().fee and
    params().n_sc_s = pre params().n_sc_s and
    params().p_min = pre params().p_min;
  -- Assuming min and max value constraints
  assert params().r_min > 1 and params().r_max >= params().r_min and
    params().fee > 0 and params().fee <= 100 and
    params().n_sc_s > 0 and params().p_min > 0;
  constant_rate = true -> pre constant_rate and rate = pre rate;
  p_seller = defaultSeller_SC -> pre c_seller;
  c_seller = update_SCSeller(p_reserve, p_sc, rate, o_msg, p_seller);
  -- Stability Property
  check "THM5"
    (constant_rate and p_seller.sold_once and o_msg.ack = RedeemedSC)
    => c_seller.price_per_sc >= p_seller.price_per_sc;
  -- Lemmas
  check (constant_rate and p_seller.sold_once)
    => p_seller.price_per_sc <= rate;
  check (constant_rate and p_seller.sold_once and p_sc > 0)
    => p_seller.price_per_sc <= min(p_reserve div p_sc, rate);
tel
```

In addition to the theorems listed in Section III, the following ones were also proved on the stablecoin specification:

```
check "THM10" o_msg.ack = MintedSC => o_msg.price = sc_price_pc;
check "THM11" (o_msg.ack = RedeemedSC and p_reserve >= p_sc * rate)
  => o_msg.price = sc_price_pc;
check "THM12" o_msg.ack = RedeemedRC
  => equity(p_reserve, p_sc, rate) > 0;
check "THM13" reserve >= 0 and n_sc >= 0 and n_rc >= 0;
check "THM14" n_rc > 0 => reserve > 0;
check "THM15" n_sc > 0 => reserve > 0;
```

Property "THM10" states that the buying price for SCs is always 1 PC in BCs, while "THM11" states that the selling price for SCs

is always 1 PC in BCs when $R \geq L(N_{SC})$. As for "THM12", it states that RCs can be sold only when $E(R, N_{SC}) > 0$. The three remaining ones are self-explanatory.

D. Improvements from Falsified Properties

Several improvements to earlier versions of Djed specification were made due to various property violations detected by BMC. Many of these improvements involved making explicit various assumptions that had previously been left implicit. The minimal parameter constraints required to guarantee stability were also validated via counterexample detection. Some of these improvements are discussed below.

Reserve Draining. In an earlier version of Djed, P_{RC}^b was specified as both buying and selling prices for RCs. It was shown that this specification can lead to a reserve draining scenario, especially when P_{RC}^t is less than P_{RC}^{min} on a selling RC action.

Theorem 7 Violation. Another reserve draining situation was detected when attempting to prove an earlier formulation of Theorem 7, which *only* assumed a constant exchange rate. The counterexample showed that there may exist a sequence of n possible actions for which $R^n < R^0$, $N_{SC}^n = N_{SC}^0$ and $N_{RC}^n = N_{RC}^0$.

Liveness issues. A deadlock situation was also detected when attempting to prove an earlier version of Djed, which was mainly related to the potential re-initialization of the stablecoin smart contract. In fact, this version was authorizing the buying of RCs only when $r(R, N_{SC}) \leq r_{max}$ was satisfied or when the buying action happened within an initialization period. However, when N_{SC} and N_{RC} were reverted back to zero after the initialization period, any order were rejected.

V. FORMALIZATION IN ISABELLE/HOL

Isabelle [5] is a generic proof assistant. It allows mathematical formulae to be expressed in a formal language and provides tools for proving those formulas in a logical calculus. The main application is the formalization of mathematical proofs and in particular formal verification of computer hardware and software. Isabelle can cope with a large class of object-logics, including several first-order logics (intuitionistic and classical), Martin-Löf's Type Theory, and Zermelo-Fraenkel set theory. Each new logic is formalized within Isabelle's meta-logic, which is an intuitionistic fragment of Church's type theory. The most widespread instance of Isabelle nowadays is Isabelle/HOL [6], which provides a higher-order logic theorem proving environment that is ready to use for big applications.

Here Isabelle/HOL was used to formalize parts of Djed's informal description as defined in Section II, including machine-checked proofs for the stability properties stated in Section III (formalization details can be found in [7]).

A. Design Considerations

We highlight the following important design decisions of our Isabelle formalization:

- *Isabelle/HOL.* Isabelle/HOL is by far the most developed object-logic, including powerful specification tools and a

vast library of formally verified mathematics, thus being the logic of choice for our work.

- *Isabelle/Isar*. Our proofs are conducted in the structured proof language Isar, allowing for proof texts naturally understandable for both humans and computers, while preserving the structure of the Djed’s informal proofs to aid the reader.
- *Locales*. We use locales (Isabelle’s module system) to bundle Djed’s parameters and their constraints. This also permits the specification of a proof context shared by all theorems, thus avoiding duplication.
- *Exact real arithmetic*. The Isabelle/HOL library includes exact real arithmetic, which we use for our formalization. This allows to formally reason at the same level of abstraction as Djed’s informal description, ignoring implementation issues such as rounding errors.
- *Transition systems*. Using Isabelle’s principle of inductive predicates, we define transition systems that model the execution of individual Djed actions and sequences thereof.
- *Notational convenience*. Isabelle provides excellent notational support: new notations can be introduced using normal mathematical symbols. We leverage such support and define special syntax to match that used by Djed’s informal description, thus improving readability.

B. Improvements of Earlier Versions of Djed

The Isabelle/HOL formalization uncovered numerous issues in earlier versions of the informal description of Djed and in earlier drafts of the informal proofs of the stability properties. The process of formalizing Djed and its stability theorems in Isabelle contributed to the more precise description and proofs that can now be found in Sections II and III. To illustrate the classes of issues uncovered by the formalization, we can enumerate the following:

- Implicit and/or missing constraints on variables/actions in the informal model: Lower and upper bounds (e.g., $fee \in [0, 1)$), non-negativity assumptions (e.g., $N_{SC} > 0$), and pre-action conditions (e.g., $n \leq N_{SC}$ for a ‘Sell n SCs’ action).
- Implicit and/or missing assumptions in the informal proofs: E.g., the exchange rates must be constant in Theorem 5, and $N_{SC} > 0$ must be an assumption in Theorem 8.
- Falsifiable formulas in the informal proofs: E.g., the conclusion of Theorem 6 had to be relaxed to $\geq$ due to impossible cases in the informal proof, which in turn caused the proof of Theorem 7 to be reworked.
- Superfluous assumptions in theorems: E.g., in Theorem 3, $P = P_{SC}^t[x]$ is redundant since it is implied by $r > 1$.
- Missing case distinctions in the informal proofs required to address specific cases that do not hold in the general case.
- Steps in the informal proofs that were missing or found to be non trivial, particularly in the proofs of Theorems 6 and 7.

- Ambiguous wording: E.g., $N_{SC} > 0$ and $N_{RC} > 0$ in the context of the statement of Theorem 7.

The issues above were uncovered in earlier versions of Djed during the course of its formal development and, thanks precisely to this formal development approach, are not present in the version of Djed and of the theorems presented here.

VI. IMPLEMENTATIONS

Djed already has the following implementations:

- **Prototype in Scala**:⁶ Djed has prototype implementation in Scala that was used for various tests and economic simulations. The prototype can be and has been used as a reference for production-ready implementations.
- **SigmaUSD on Ergo**:⁷ Djed was implemented in ErgoScript. It was deployed with the name SigmaUSD in Q1 2021 on Ergo, and its success is evidenced by the following indicators: SigmaUSD’s reserve captured 6% of the entire supply of ERG; the stablecoin market cap quickly rose to a peak of 10 million USD; there were approximately 30 transactions per day buying or selling stablecoins or reservecoins, amounting to approximately 1% of the total number of transactions per day on Ergo; the reserve ratio is being kept consistently at healthy levels between 300% and 400%; and the stablecoin was able to maintain its peg during the market crashes of May 2021, when the price of ERG (along with most other cryptocurrencies) fell 44% in 1 day and 60% in 5 days, and also of May 2022.
- **Implementation in Plutus for Cardano**: An implementation of Djed in the Plutus smart contract language [8]–[10] is available⁸. This version uses monetary policies [11] and native assets [12] to represent stablecoins and reservecoins. Another implementation in Plutus exists and has been deployed⁹ to Cardano in Q1 2023.
- **Implementation in Solidity for EVM-Compatible Blockchains**: Djed was implemented in Solidity three times. The first two follow the protocol exactly as described here and differ from each other in that one has as basecoin the native currency of the blockchain whereas the other takes as basecoin any ERC20-compliant token. The third one has additional features, such as transaction limits, treasury and UI fees, and the simultaneous sale of SCs and RCs. It has been deployed¹⁰ to the testnet of Milkmeda-C1, a sidechain of Cardano.

VII. RELATED WORK

The three stablecoin protocols that were most influential in the design of Djed were: Seigniorage Shares, DAI and Staticoin. The following subsections describe, qualitatively, the similarities and differences between these protocols and Djed.

⁶<https://github.com/input-output-hk/djed-stablecoin-prototype>

⁷<https://sigmausd.io/> and <https://github.com/Emurgo/age-usd>

⁸<https://github.com/input-output-hk/plutus-apps/tree/main/plutus-use-cases/src/Plutus/Contracts/Stablecoin.hs>

⁹<https://www.djed.xyz>

¹⁰<https://milkmeda-c1-testnet.djed.one>

Seigniorage Shares: One of the earliest examples of algorithmic stablecoin is the *seigniorage shares* protocol [13]. Its stabilization mechanisms are similar to the mechanisms used by most central banks. The protocol aims to adjust the circulating supply. If the price is too high, the circulating supply will be increased. If it is too low, the circulating supply will be decreased. To take stablecoins out of circulation, users are encouraged to sell their stablecoins in exchange for seigniorage shares; conversely, when the supply needs to be increased, stablecoins are issued and distributed to seigniorage share holders. In this way, the seigniorage (i.e. the revenue from printing money) goes to seigniorage share holders. While this mechanism may seem convincing at first, the collapse in May 2022 of the Terra stablecoin protocol, a variant of Seigniorage Shares, shows this mechanism is insufficient for stability. The value of a seigniorage share depends on the stability of the protocol and on the expectation of growth of the stablecoin supply. Therefore, it is unreasonable to think that seigniorage shares provide any “backing” to the stablecoins, since this would constitute a circular dependency. In contrast, Djed stablecoins are *backed* by an independent asset (the BCs) with a value that, although volatile, is unrelated to the Djed protocol. Furthermore, Djed’s revenue comes from market making fees instead of seigniorage.

DAI: Crypto-collateralized stablecoins, of which DAI [14] is the most prominent example, are similar to crypto-backed stablecoins, with the subtle but important difference, however, that cryptocurrencies are used to provide collateral for stablecoin-denominated loans instead of reserves. Similarly to how a house that serves as collateral in a mortgage does not count as reserves in the bank that is providing the loan, the collateral provided for the issuance of a crypto-collateralized stablecoin does not count as reserve. Due to the lack of reserves, pure crypto-collateralized stablecoins resemble an extreme form of fractional reserve banking (with a fraction of 0%). Unlike commercial bank money (i.e. money in a bank account), which can be exchanged at any time by anyone holding it for central bank money by withdrawing cash from the bank’s reserves, crypto-collateralized stablecoins cannot be exchanged for anything by withdrawing from the stablecoin contract’s reserves, simply because there are no reserves. When interacting with the contract, users can only use stablecoins to repay their own loans and recover their own collateral or to bid in auctions for liquidated collateral from others.

Staticoin: The Staticoin protocol [15] has a *riskcoin* whose function is very similar to Djed’s reservecoin. The main difference, from a user’s perspective, is that riskcoin’s appeal rests in the leverage it provides with respect to base coins, whereas reservecoin’s appeal rests mostly in the market making revenue through fees, even though it also provides some leverage. Djed also has minimum and maximum reserve ratio thresholds, which protect stablecoin holders from market crashes and reservecoin holders from dilution, whereas Staticoin does not.

VIII. CONCLUSIONS AND FUTURE WORK

Djed is an autonomous crypto-backed stablecoin protocol that strives to keep a reserve ratio significantly greater than one in order to guarantee the peg. This was an intentionally conservative design decision, because stability is of paramount importance and achieving stability with fractional reserves or even with no reserve (or collateralization) at all appears to be a much more elusive goal. This decision paid off. As shown in this paper, Djed is not only provably stable in theory but its implementation and deployment on Ergo, which survived the market crashes of 2021 and 2022 with the peg unscathed, has already provided strong empirical evidence of its stability in practice as well. In contrast, many recent stablecoin projects that attempted to achieve stability with fractional reserves (e.g. FEI ¹¹ and Iron ¹²) or with no reserves (e.g. Terra) failed.

The design of Djed is the result of a non-trivial compromise between attempting to satisfy many desirable stability properties and taming complexity. The outcome is an intentionally simple solution that is fully defined in just 1 page (cf. Section II). Thanks to its simplicity, the protocol is highly amenable to the use of formal methods, such as model checking and interactive theorem proving. It took only roughly 4 weeks of work to obtain the results discussed in Sections IV and V. Therefore, the work presented here can also be considered a success story for the discipline of formal methods, which has a reputation of being time-consuming.

Based on our observations of the deployment of the Djed protocol on Ergo, key directions for future work include increasing the robustness against potential oracle delays or price foresight and against large transactions that change the reserve ratio significantly. An extended version of this paper [7] already discusses some variations of the protocol that are worth exploring further.

REFERENCES

- [1] A. Champion, A. Mebsout, C. Stickse, and C. Tinelli, “The Kind 2 Model Checker,” in *Computer Aided Verification*. Springer International Publishing, 2016, pp. 510–517.
- [2] N. Halbwachs, P. Caspi, P. Raymond, and D. Pilaud, “The synchronous data flow programming language LUSTRE,” in *Proceedings of the IEEE*, vol. 79, no. 9, 1991, pp. 1305–1320.
- [3] A. Biere, “Bounded Model Checking,” in *Handbook of Satisfiability*, ser. Frontiers in Artificial Intelligence and Applications. IOS Press, 2009, vol. 185, pp. 457–481.
- [4] N. Een and N. Sörensson, “Temporal Induction by Incremental SAT Solving,” *Electronic Notes in Theoretical Computer Science*, vol. 89, pp. 543–560, December 2003.
- [5] L. Paulson and T. Nipkow, *Isabelle: A Generic Theorem Prover*, ser. Lecture Notes in Computer Science. Springer, 1994. [Online]. Available: <https://books.google.com.ar/books?id=RxlhQG0-cGwC>
- [6] T. Nipkow, L. C. Paulson, and M. Wenzel, *Isabelle/HOL – A Proof Assistant for Higher-Order Logic*, ser. LNCS. Springer, 2002, vol. 2283.
- [7] J. Zahnentferner, D. Kaidalov, J.-F. Etienne, and J. Díaz, “Djed: A formally verified crypto-backed pegged algorithmic stablecoin,” Cryptology ePrint Archive, Report 2021/1069, 2021, <https://eprint.iacr.org/2021/1069>.

¹¹<https://www.coindesk.com/1b-fei-stablecoins-rocky-start-is-a-wake-up-call-for-defi-investors>

¹²<https://cointelegraph.com/news/iron-finance-bank-run-stings-investors-a-lesson-for-all-stablecoins>

- [8] “The Plutus Platform.” [Online]. Available: <https://developers.cardano.org/docs/smart-contracts/plutus/>
- [9] M. P. Jones, V. Gkoumas, R. Kireev, K. MacKenzie, C. Nester, and P. Wadler, “Unraveling Recursion: Compiling an IR with Recursion to System F,” in *Mathematics of Program Construction - 13th International Conference, MPC 2019, Porto, Portugal, October 7-9, 2019, Proceedings*, ser. Lecture Notes in Computer Science, G. Hutton, Ed., vol. 11825. Springer, 2019, pp. 414–443. [Online]. Available: https://doi.org/10.1007/978-3-030-33636-3_15
- [10] J. Chapman, R. Kireev, C. Nester, and P. Wadler, “System F in Agda, for Fun and Profit,” in *Mathematics of Program Construction - 13th International Conference, MPC 2019, Porto, Portugal, October 7-9, 2019, Proceedings*, ser. Lecture Notes in Computer Science, G. Hutton, Ed., vol. 11825. Springer, 2019, pp. 255–297. [Online]. Available: https://doi.org/10.1007/978-3-030-33636-3_10
- [11] J. Zahnentferner, “Multi-Currency Ledgers,” Cryptology ePrint Archive, Report 2020/895, 2020, <https://eprint.iacr.org/2020/895>.
- [12] M. M. T. Chakravarty, J. Chapman, K. MacKenzie, O. Melkonian, J. Müller, M. P. Jones, P. Vinogradova, P. Wadler, and J. Zahnentferner, “Utxo_{sf} ma: UTXO with multi-asset support,” in *Leveraging Applications of Formal Methods, Verification and Validation: Applications - 9th International Symposium on Leveraging Applications of Formal Methods, ISOFA 2020, Rhodes, Greece, October 20-30, 2020, Proceedings, Part III*, ser. Lecture Notes in Computer Science, T. Margaria and B. Steffen, Eds., vol. 12478. Springer, 2020, pp. 112–130. [Online]. Available: https://doi.org/10.1007/978-3-030-61467-6_8
- [13] R. Sams, “A Note on Cryptocurrency Stabilisation: Seigniorage Shares,” [Online], 2014, <https://blog.bitmex.com/wp-content/uploads/2018/06/A-Note-on-Cryptocurrency-Stabilisation-Seigniorage-Shares.pdf>.
- [14] “The Maker Protocol: MakerDAO’s Multi-Collateral Dai (MCD) System,” [Online], 2017, <https://makerdao.com/en/whitepaper/>.
- [15] P. Edge, “statiCoin and riskCoin,” [Online], 2017, <https://github.com/genkifs/staticoin/blob/master/Staticoin%20-%20Whitepaper.pdf>.

APPENDIX A PROOF OF THEOREM 6

Theorem 6 (Monotonically Increasing Equity per Reserve-coin). *Assuming that the exchange rate remains constant and $N_{RC} > 0$, for every action a , $\frac{E(R^a, N_{SC}^a)}{N_{RC}^a} \geq \frac{E(R, N_{SC})}{N_{RC}}$, where R^a , N_{SC}^a , $N_{RC}^a > 0$ are respectively, the reserve, number of stablecoins and number of reservecoins after action a .*

Proof: There are 4 cases:

- **Buy n SCs.** In this case, $\frac{E(R^a, N_{SC}^a)}{N_{RC}^a} = \frac{R+n(1+fee)P_{SC}-(N_{SC}+n)P_{SC}^a}{N_{RC}}$. By the assumption of constant exchange rate and given that $P_{SC}^a = P_{SC}$ (due to restriction of the buying operation if $P_{SC} < P_{SC}^t$) we have that $\frac{E(R^a, N_{SC}^a)}{N_{RC}^a} = \frac{R-N_{SC}P_{SC}+nfeeP_{SC}}{N_{RC}} > \frac{R-N_{SC}P_{SC}}{N_{RC}} = \frac{E(R, N_{SC})}{N_{RC}}$.
- **Sell n SCs.** In this case, $\frac{E(R^a, N_{SC}^a)}{N_{RC}^a} = \frac{R-n(1-fee)P_{SC}-(N_{SC}-n)P_{SC}^a}{N_{RC}}$. It is easy to see that inequality $\frac{R-n(1-fee)P_{SC}-(N_{SC}-n)P_{SC}^a}{N_{RC}} \geq \frac{R-N_{SC}P_{SC}}{N_{RC}} = \frac{E(R, N_{SC})}{N_{RC}}$ holds if $n(1-fee)P_{SC} + (N_{SC}-n)P_{SC}^a \leq N_{SC}P_{SC}$. We will prove the latter for 2 different cases:
 - If $P_{SC} = \frac{R}{N_{SC}} \leq P_{SC}^t$: given that $P_{SC}^a \leq \frac{R^a}{N_{SC}^a}$, we can define $P_{SC}^a = \frac{R^a}{N_{SC}^a}\gamma$ for some $\gamma \in (0, 1]$. Provided that $R^a = R - n(1-fee)\frac{R}{N_{SC}}$ and doing substitutions we have: $n(1-fee)\frac{R}{N_{SC}} + (N_{SC}-n)\frac{R-n(1-fee)\frac{R}{N_{SC}}}{N_{SC}}\gamma \leq N_{SC}\frac{R}{N_{SC}}$,

$$\frac{n(1-fee)}{N_{SC}}(1-\gamma) \leq 1-\gamma,$$

which holds due to $N_{SC} > n(1-fee)$.

- If $P_{SC} < \frac{R}{N_{SC}}$: first let’s show that $\frac{R}{N_{SC}} < \frac{R^a}{N_{SC}^a}$. Given that $P_{SC} < \frac{R}{N_{SC}}$ we can define $P_{SC} = \frac{R}{N_{SC}}\sigma$ for some $\sigma \in (0, 1)$. Hence, $\frac{R^a}{N_{SC}^a} = \frac{R-n(1-fee)\frac{R}{N_{SC}}\sigma}{N_{SC}-n}$. Then we have:

$$\frac{R}{N_{SC}} < \frac{R-n(1-fee)\frac{R}{N_{SC}}\sigma}{N_{SC}-n}, \quad 1 < \frac{N_{SC}-n(1-fee)\sigma}{N_{SC}-n},$$

which is true due to $n > n(1-fee)\sigma$.

From $\frac{R}{N_{SC}} < \frac{R^a}{N_{SC}^a}$ follows that $P_{SC} = P_{SC}^a = P_{SC}^t$, hence we have that:

$$n(1-fee)P_{SC}^t + (N_{SC}-n)P_{SC}^t \leq N_{SC}P_{SC}^t,$$

which holds due to $N_{SC}-n+n(1-fee) \leq N_{SC}$.

- **Buy n RCs.** First let’s show that $R < R^a$. It is easy to see given that $R^a = R + n(1+fee)P_{RC}^b$, where $P_{RC}^b \geq P_{RC}^{min} > 0$. Then, by definition of P_{SC} , it holds that $P_{SC} = \min(P_{SC}^t, \frac{R}{N_{SC}}) \leq P_{SC}^a = \min(P_{SC}^t, \frac{R^a}{N_{SC}^a})$. Now we will prove the theorem for 2 different cases:

- If $P_{SC} = \frac{R}{N_{SC}} \leq P_{SC}^t$: in this case $P_{SC}^a \leq \frac{R^a}{N_{SC}^a}$, hence we can define $P_{SC}^a = \frac{R^a}{N_{SC}^a}\gamma$ for some $\gamma \in (0, 1]$. Note that from $P_{SC} = \frac{R}{N_{SC}}$ follows that $E(R, N_{SC}) = R - N_{SC}\frac{R}{N_{SC}} = 0$. Making substitutions we have:

$$\frac{E(R^a, N_{SC}^a)}{N_{RC}^a} = \frac{R^a - N_{SC}^a P_{SC}^a}{N_{RC}^a} \geq \frac{E(R, N_{SC})}{N_{RC}},$$

$$\frac{(R+n(1+fee)P_{RC}^b)(1-\gamma)}{N_{RC}+n} \geq 0,$$

which holds due to $(1-\gamma) \geq 0$.

- If $P_{SC} < \frac{R}{N_{SC}}$: in this case $P_{SC} = P_{SC}^a = P_{SC}^t$. By definition $P_{RC}^b \geq \frac{R-N_{SC}P_{SC}}{N_{RC}}$ and, given that $R - N_{SC}P_{SC} > 0$, we can define $P_{RC}^b = \frac{R-N_{SC}P_{SC}}{N_{RC}}\alpha$ for some $\alpha \geq 1$. Making substitutions we have $\frac{E(R^a, N_{SC}^a)}{N_{RC}^a} = \frac{R+n(1+fee)P_{RC}^b - N_{SC}P_{SC}}{N_{RC}+n}$ and $\frac{E(R, N_{SC})}{N_{RC}} = \frac{R-N_{SC}P_{SC}}{N_{RC}}$, hence:

$$\frac{R-N_{SC}P_{SC}}{N_{RC}+n} + \frac{n(1+fee)\frac{R-N_{SC}P_{SC}}{N_{RC}}\alpha}{N_{RC}+n} \geq \frac{R-N_{SC}P_{SC}}{N_{RC}},$$

$$\frac{N_{RC}}{N_{RC}+n} + \frac{n(1+fee)\alpha}{N_{RC}+n} \geq 1, \quad \frac{N_{RC}+n(1+fee)\alpha}{N_{RC}+n} \geq 1,$$

which holds given that $\alpha \geq 1$.

- **Sell n RCs.** Given that $R^a = R - n(1-fee)P_{RC}^t$, where $P_{RC}^t = \frac{R-N_{SC}P_{SC}}{N_{RC}} \geq 0$, it follows that $R \geq R^a$. Then, by definition of P_{SC} , it holds that $P_{SC} = \min(P_{SC}^t, \frac{R}{N_{SC}}) \geq P_{SC}^a = \min(P_{SC}^t, \frac{R^a}{N_{SC}^a})$. Note that the Sell RC operation is allowed only if $r(R^a, N_{SC}^a) \geq r_{min}$, hence $P_{SC}^a = P_{SC}^t$ and $P_{SC} = P_{SC}^t$. Making substitutions we get: $\frac{E(R^a, N_{SC}^a)}{N_{RC}^a} = \frac{R-n(1-fee)P_{RC}^t - N_{SC}P_{SC}^t}{N_{RC}-n}$ and $\frac{E(R, N_{SC})}{N_{RC}} = \frac{R-N_{SC}P_{SC}^t}{N_{RC}}$, hence:

$$\frac{R-N_{SC}P_{SC}^t}{N_{RC}-n} - \frac{n(1-fee)\frac{R-N_{SC}P_{SC}^t}{N_{RC}}}{N_{RC}-n} \geq \frac{R-N_{SC}P_{SC}^t}{N_{RC}},$$

$$\frac{N_{RC}}{N_{RC}-n} - \frac{n(1-fee)}{N_{RC}-n} \geq 1, \quad \frac{N_{RC}-n(1-fee)}{N_{RC}-n} \geq 1,$$

which holds due to $n(1-fee) \leq n$. ■